

1. Executive Framing

Presenter: Bharat

Role context: Senior Engineering Manager, Agent Platform

Audience: Peer Engineering Managers, Architects, VP Engineering

Problem

Customers need a reliable, actionable view of agent operational health across millions of endpoints. Today, health signals are fragmented, noisy, and partially dependent on legacy heartbeat logic that is already creating customer escalations.

Product Outcome

Build Singularity Health Center as a real-time operational intelligence platform for endpoint health.

Success Metrics

- Detect high-impact health anomalies within 5 minutes.
- Serve dashboard APIs at less than 200 ms p95 for common tenant queries.
- Reduce false "Offline" escalations by 50 percent within two quarters.
- Provide replay and backfill for missed events.
- Establish reusable alerting and coalescing capabilities for future agent-platform use cases.

Leadership Position

This is not only a feature delivery. It is a platform modernization effort that must land while we protect customers from current operational pain.

2. Requirements And Assumptions

MVP Functional Scope

- Detect agent offline/connectivity loss, agent disabled, anti-tamper disabled, and low disk/resource risk.
- Coalesce repeated events into actionable alerts.
- Provide tenant-scoped dashboard, alert list, drill-down, and APIs.
- Support alert lifecycle: open, acknowledged, resolved, suppressed.
- Run on allowlisted tenants before broad rollout.

Non-Functional Requirements

- Billions of events per day, millions of agents.
- Multi-tenant isolation and fair usage controls.
- Less than 5 minute detection freshness.
- Less than 200 ms p95 API latency for dashboard read paths.
- At-least-once event delivery with idempotent processing.
- Replay, backfill, and auditable alert decisions.

Key Assumptions

- Central Ingestion Gateway remains the long-term telemetry entry point.
- Health Center owns event processing, alert state, APIs, and UI.
- Legacy heartbeat logic remains customer-facing until Health Center has shadow validation.

3. High-Level Architecture

```
Agents
|
Central Ingestion Gateway
|
Durable Event Bus
Kafka / PubSub / Kinesis-equivalent
|
Stream Processing
Flink / Kafka Streams-equivalent
|
Rules + Coalescing Engine
|
Operational Alert Store
DynamoDB / Cassandra / Bigtable-equivalent
|
Read Model + Search Index
OpenSearch / Elasticsearch-equivalent
|
Health Center APIs
|
SentinelOne Console UI
```

Design Principle

Separate the write-heavy real-time path from the read-optimized dashboard path. This protects ingestion and detection from UI query patterns while keeping the console fast.

Reusable Platform Capabilities

- Health telemetry contract and schema governance.
- Stream coalescing library/service.
- Alert lifecycle service.
- Rule evaluation framework.
- Tenant rollout, replay, and backfill tooling.

4. Event Processing And Alert Coalescing

Recommended Approach

Use stream processing for real-time anomaly detection and alert coalescing.

Why

- The 5 minute freshness requirement is explicit.
- Raw event volume makes database-first aggregation expensive and operationally risky.
- Stream-time windows naturally model examples such as "50 anti-tamper disabled events from the same agent in 10 minutes."
- Durable event logs allow replay when rules change or processors fail.

Coalescing Model

- Partition by tenant and agent ID.
- Maintain keyed state for active anomaly windows.
- Emit alert transitions rather than every raw event.
- Store raw event references for audit/debugging, not for every UI query.

Trade-Off

This adds platform complexity, but it moves complexity into a purpose-built layer where latency, scale, and replay are controllable.

5. Storage And API Read Path

Storage Roles

- Durable event bus: short to medium retention, replay, and consumer decoupling.
- Operational alert store: current alert state, lifecycle, idempotency, tenant/agent keys.
- Search/read model: low-latency filtering, aggregation, and dashboard queries.
- Data lake: long-term retention, analytics, audits, model/rule tuning.

API Strategy

- Read from precomputed health views, not raw telemetry.
- Cache tenant-level dashboard summaries with freshness metadata.
- Use cursor pagination for alert lists.
- Bound query shapes to protect p95 latency.
- Degrade gracefully if search lags by serving current alert state with an explicit freshness indicator.

Latency Budget

- API gateway/auth: 20-40 ms.
- Service orchestration: 40-60 ms.
- Read store/search: 70-90 ms.
- Serialization/network buffer: 20-30 ms.

6. Reliability And Sev-1 Response

Health Center SLOs

- Event processing freshness: 99 percent of events processed within 5 minutes.
- Dashboard API latency: less than 200 ms p95 for standard queries.
- Alert delivery correctness: no silent processing gaps longer than 5 minutes.
- Data loss objective: no acknowledged event loss after ingestion into durable bus.

Detection Controls

- End-to-end synthetic health events per tenant cohort.
- Ingestion rate, processing lag, dropped-event, and alert freshness monitors.
- Stage-by-stage dashboards: gateway, event bus, processors, stores, APIs.
- Canary processors and feature-flagged rollouts.

Sev-1: Silent Processing Stop

1. Declare incident, assign incident commander, customer/comms lead, and technical leads.
2. Determine blast radius by tenant, region, and pipeline stage.
3. Compare gateway ingress, event-bus offsets, processor lag, and store writes.
4. Mitigate by failing over consumers, disabling bad rules, or pausing rollout.
5. Replay from durable event log and backfill missed alerts.
6. Publish customer-impact assessment and complete a blameless postmortem.

7. Roadmap: MVP To GA

Quarter 1: Foundations And MVP

- Define telemetry contracts and schema ownership with Ingestion Gateway.
- Build ingestion adapter, stream processor skeleton, alert store, and basic APIs.
- Implement top four anomaly types and basic coalescing.
- Build initial dashboard and tenant allowlist rollout.
- Add SLO dashboards, synthetic events, and runbooks from day one.

Quarter 2: GA Readiness

- Harden scale, replay, idempotency, and backfill.
- Add alert lifecycle, suppression, richer drill-down, and search filters.
- Shadow-compare Health Center offline status against legacy heartbeat service.
- Run load tests, chaos drills, and phased production expansion.

Quarter 3: Platformization

- Migrate selected offline source-of-truth decisions from legacy heartbeat.
- Expand rule catalog and reusable coalescing framework.
- Add historical trends, reporting, and notification integrations.
- Formalize platform APIs for other SentinelOne health use cases.

8. Dependency Delay: Ingestion Gateway

Scenario

The core Ingestion Gateway team is delayed by two months and cannot route new health telemetry on the original timeline.

Response

- Do not idle the team.
- Freeze an explicit telemetry contract and build an adapter boundary.
- Generate synthetic and replayed telemetry streams to unblock processors, APIs, UI, and QA.
- Use existing heartbeat/agent metadata streams for a reduced MVP where possible.
- Negotiate a thin gateway integration first: route minimal high-value signals before the full contract.
- Move GA dates only for dependency-bound scope, not for all downstream development.

Leadership Message

The plan absorbs dependency risk by decoupling interfaces, validating downstream systems early, and preserving customer-visible progress.

9. Operational Drain: Legacy Offline Escalations

Scenario

Support escalations rose 40 percent because agents are incorrectly showing as "Offline" in the console. The status logic depends on a legacy heartbeat service the team still owns.

Response

- Treat this as customer trust risk, not background maintenance.
- Create a short-lived stabilization lane with 2 engineers plus QA support.
- Keep 6 engineers focused on Health Center MVP, with one tech lead coordinating interfaces and risk.
- Add targeted instrumentation, identify false-offline causes, and ship a narrow patch.
- Use findings to improve Health Center offline semantics and migration tests.

Allocation

- Legacy stabilization: 2 engineers.
- Health Center build: 6 engineers.
- QA/release validation: 1 engineer.
- Tech lead/SEM coordination: 1 engineer.

Decision Rule

Escalate allocation only if the legacy issue creates active Sev-1/Sev-2 customer impact across major tenants.

10. Leadership, Risks, And Outcomes

Alert Coalescing Conflict

- Establish decision criteria: freshness, scale, cost, operability, delivery risk.
- Run a one-week design spike with realistic load and failure scenarios.
- Make the decision using evidence, not seniority or preference.
- Commit as one team, document trade-offs, and revisit only if assumptions change materially.

Major Risks

- Gateway dependency slips.
- Stream processing complexity impacts delivery.
- Legacy escalations consume roadmap capacity.
- Dashboard query patterns threaten API latency.
- Tenant-specific noise creates alert fatigue.

Mitigations

- Adapter contracts, synthetic streams, and phased integration.
- Start with narrow rules and reusable framework, not a generic rule engine.
- Dedicated stabilization lane with explicit exit criteria.
- Precomputed read models and bounded queries.
- Coalescing, suppression, tenant rollout, and feedback loops.

Expected Outcomes

- Faster and more trustworthy agent-health visibility.
- Reduced support burden from false offline states.
- A durable platform foundation for future operational health use cases.